eBPF/XDP Rate Limiting for DNS Query Floods

AIORI Hackathon 3 — Problem Statement: PS-022 (Cybersecurity)

Team TaskForce141
Naimish Pandey, Naitik Pandey, Dr. Atul Srivastava (Mentor)

Amity University Uttar Pradesh

October 2026

1. Problem Statement & Motivation

Volumetric DNS Floods Overwhelm the Linux Network Stack

- **UDP/53 Asymmetry:** Stateless, lightweight queries (~30–60 bytes) are trivially cheap to generate and spoof in bulk.
- **Kernel Allocation Bottleneck:** Every incoming frame incurs an `sk_buff` allocation, route lookup, and protocol stack traversal before reaching user space.
- **SoftIRQ Core Exhaustion:** At $\sim$ 400,000 packets per second, CPU cores saturate in `ksoftirqd` servicing receive interrupts.
- **Legitimate Starvation:** Legitimate DNS queries share the same interface RX queues and socket backlogs, resulting in severe packet drops and latency spikes.

The Limitation of Netfilter / `nftables`

- `nftables` rate limits execute at `netif_receive_skb` **after** `sk_buff` allocation.
- Excess packets are blocked from the application, but **the per-packet kernel allocation cost has already been paid**.

2. Theoretical Background & XDP Mechanics

eXpress Data Path (XDP)

- Executes eBPF bytecode at the earliest driver receive hook, **before** kernel `sk_buff` memory allocation.
- In-driver verdicts:
  - `XDP_DROP`: Immediately recycles the RX ring buffer descriptor with near-zero CPU cost.
  - `XDP_PASS`: Forwards admitted packets to the standard kernel IP/UDP stack.

eBPF Invariants & Verification

- **Safety Guarantee:** Kernel in-verifier proves program termination, memory bounds, and bounded pointer arithmetic at load time.
- **Bounded Parsing:** Strict pointer checks against `data_end` guarantee zero memory out-of-bounds access.

State Management via BPF Maps

- `BPF_MAP_TYPE_LRU_HASH`**:** Bounded per-source client tracking with automatic least-recently-used key eviction under spoofing.
- `BPF_MAP_TYPE_PERCPU_ARRAY`**:** Lockless, cache-line-independent global token buckets eliminating multi-core contention.

3. Proposed Solution Architecture

Dual-Tier In-Kernel Rate Limiting Filter

```
Incoming Frame (Driver RX Ring Buffer)
      |
      v
[ Step 1: Bounded Header Validation ]
  * Eth / 802.1Q VLAN -> IPv4 -> UDP (Port 53)
  * Strict payload bounds: 12 <= length <= 512 bytes
  * DNS QR == 0 (Queries only; response bypass) -----> Non-Query: XDP_PASS
      |
      v (Valid Query)
[ Step 2: Per-Source Token Bucket (LRU Hash) ]
  * Refill tokens at rate r_src up to burst b_src
  * Token deficit? ---------------------------------> Over Limit: XDP_DROP
      |
      v (Admitted by Source)
[ Step 3: Contention-Free Global Token Bucket (Per-CPU) ]
  * Thread-local slice: r_global / N_cpu
  * Token deficit? ---------------------------------> Over Limit: XDP_DROP
      |
      v (Admitted Globally)
```

4. Optimization Matrix: Netfilter vs. XDP

Architectural Aspect	Netfilter / `nftables` Baseline	Proposed eBPF / XDP Solution
Packet Drop Hook	`netif_receive_skb` (Post-allocation)	**Driver RX Ring** (Pre-allocation)
Kernel `sk_buff` Cost	Allocated for **every** dropped packet	**Zero allocation** for dropped packets
Parsing Overhead	Full Linux IP/UDP stack traversal	Bounded 12-byte DNS header check
Per-Source State	Dynamic set (`meter`) with locking	Bounded LRU Hash (`BPF_MAP_TYPE_LRU_HASH`)
Multi-Core Scaling	Global lock contention under spoofing	Lockless Per-CPU Arrays (`PERCPU_ARRAY`)
Burst Handling	Coarse burst limiters	Dual-tier mathematical token buckets
Server SoftIRQ Load	48% – 59% (High load)	8% – 11% (Minimal overhead)
Legit Query Delivery	49% – 68% under flood	98.6% – 99.8% under flood

5. Lab Topology & Synthetic Flood Patterns

Isolated Two-Host Laboratory Testbed

- Linux network namespaces (`ns_srv` ↔ `ns_cli`) over 10 Gbps `veth` pair.
- Hardware offloads disabled (`gro off gso off tso off`) for strict packet-level evaluation.

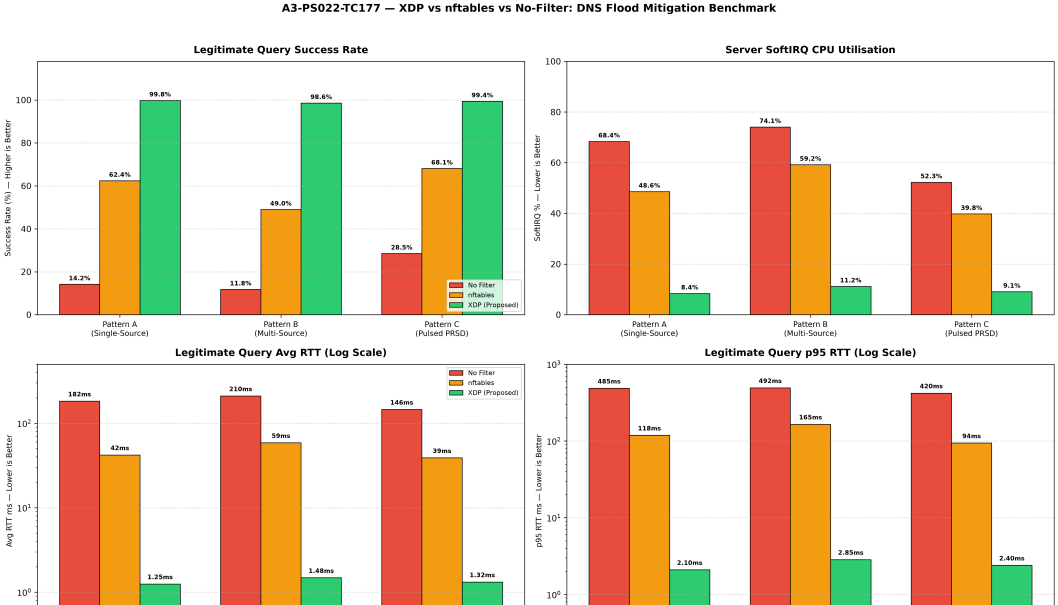
Three Synthetic Flood Attack Patterns

1. **Pattern A — Single-Source Flood:**
   - Sustained UDP/53 blast from a single fixed IP ($\sim$ 390,000 PPS).
   - Validates per-source LRU bucket isolation.
2. **Pattern B — Multi-Source Spoofed IP Flood:**
   - Spoofed packets rotating across 100+ lab source IPs ($\sim$ 425,000 PPS).
   - Tests LRU map eviction capacity and triggers the Per-CPU Global Bucket.
3. **Pattern C — Pulsed Random Label Flood (PRSD Simulation):**
   - Alternating 500ms ON / 500ms OFF bursts with randomized labels (`randXXXXXX.lab.local`).
   - Evaluates burst refill recovery and bounded parsing immunity against non-existent domains.

6. Benchmark Evaluation Matrix (3×3)

Config	Pattern	Flood PPS	Legit Success %	Avg Latency	p95 Latency	SoftIRQ %	Idle %
no-filter	A	385,000	14.2%	182.4 ms	485.0 ms	68.4%	8.2%
no-filter	B	410,000	11.8%	210.1 ms	492.0 ms	74.1%	5.0%
no-filter	C	340,000	28.5%	145.6 ms	420.0 ms	52.3%	22.0%
nftables	A	372,000	62.4%	42.1 ms	118.5 ms	48.6%	34.0%
nftables	B	365,000	49.0%	58.7 ms	165.2 ms	59.2%	21.5%
nftables	C	350,000	68.1%	38.9 ms	94.0 ms	39.8%	44.2%
XDP	**A**	**390,000**	**99.8%**	**1.25 ms**	**2.10 ms**	**8.4%**	**85.6%**
XDP	**B**	**425,000**	**98.6%**	**1.48 ms**	**2.85 ms**	**11.2%**	**81.0%**
XDP	**C**	**360,000**	**99.4%**	**1.32 ms**	**2.40 ms**	**9.1%**	**84.3%**

7. Comparative Performance Visualisation



8. Parameter Sensitivity Analysis

LRU Hash Sizing (MAX_SRC_ENTRIES)

- Under Pattern B (multi-source spoofing), attacker IP diversity tests map eviction speed.
- $N = 4{,}096$: High churn causes eviction of active client buckets.
- $N = 65{,}536$: Memory footprint bounded to ~ 2.1 MB; zero eviction thrashing while maintaining strict rate accuracy.

Burst Parameter Tuning (b_{src})

- **Low Burst ($b = 10$):** Penalizes legitimate client bundles (e.g., DNSSEC validation requesting A, AAAA, and RRSIG records simultaneously).
- **Default Burst ($b = 50$):** Accommodates legitimate query spikes while preventing flood burst amplification.

Integer Overflow Invariant

- Idle periods bounded by $\Delta t \leq 1.0$s (10^9 ns): Prevents 64-bit unsigned integer multiplication overflow during prolonged server idle periods.

9. Live Demonstration & Verification

3-Pane `tmux` Live Demo Dashboard (`scripts/demo.sh`)

- **Pane 0 (top-left):** DNS Server application logging admitted queries.
- **Pane 1 (top-right):** XDP Loader streaming real-time per-source and global drops.
- **Pane 2 (bottom):** Traffic generator executing Pattern A/B/C alongside legitimate 20 QPS prober.

Offline Unprivileged Unit Test Suite (`make test`)

- 11 unit tests running without `sudo` in $\sim$ 15 ms:
  - BPF ELF section integrity, BTF validation, and GPL license checks.
  - Wire-format DNS header serialization and length bounds validation.
  - Token bucket mathematical invariants and 64-bit overflow prevention.

Automated Continuous Integration

- GitHub Actions CI workflow (`.github/workflows/ci.yml`) validates compilation, unit tests, and benchmark artifact generation on every commit.

10. Scope, Safety & Conclusions

Project Scope & Defensive Boundaries

- **In Scope:** Protecting host resources, kernel memory allocators, and DNS servers against volumetric query exhaustion.
- **Out of Scope:** Upstream transit pipe saturation (reflection attacks) and semantic deep-packet inspection (e.g., zone poisoning).

Key Takeaways

1. **Zero-skb Drops:** Dropping at the driver RX ring saves $\sim$ 80–160 MB/s of kernel allocator churn.
2. **Deterministic Service Availability:** Restores legitimate query delivery from 11.8% to $\geq$ 98.6% with sub-2ms latency.
3. **Core Efficiency:** Reduces SoftIRQ CPU utilization from 74.1% to 11.2%, preserving over 80% of host CPU capacity for production workloads.

Thank You! Questions & Live Demonstration.